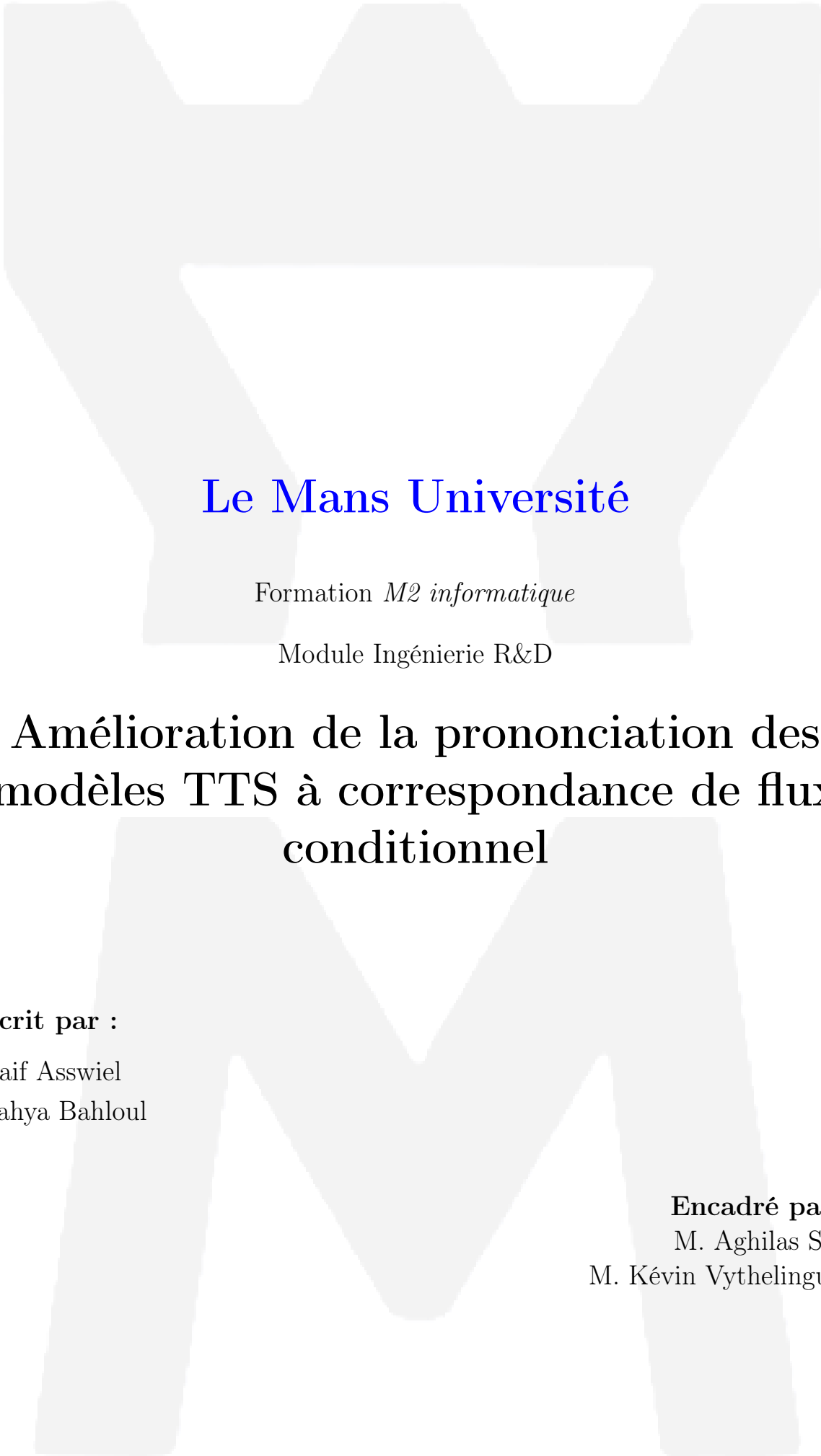




Le Mans Université

Formation *M2 informatique*

Module Ingénierie R&D

Amélioration de la prononciation des modèles TTS à correspondance de flux conditionnel

Ecrit par :

Naif Asswiel

Yahya Bahloul

Encadré par :

M. Aghilas Sini

M. Kévin Vythelingum

Table des matières

1	Introduction	2
2	Présentation du projet	2
3	Contexte et Problématique	2
4	Jeu de données et prétraitement	3
5	Modèles de Flow-matching – ZipVoice	3
5.1	Concept	3
5.2	Évaluation des limites sur Blizzard2023	4
5.2.1	Métriques	4
5.2.2	Résultats et analyse	5
5.3	Analyse du Fine-tuning de ZipVoice	5
5.3.1	Échec de l’Approche Généraliste (V1)	5
5.3.2	Succès de la Spécialisation (V2)	6
5.3.3	Dynamique de la Perte	6
5.4	Validation de la Métrique : Fine-tuning de Whisper avec LoRA	6
5.5	Limitations	7
6	Optimisation de la sélection de prompt par apprentissage par renforcement	7
6.1	Motivation	7
6.2	Prétraitement	7
6.3	Architecture du pipeline	7
6.4	Résultats et analyse	10
6.5	Limitations	11
6.6	Écueils rencontrés et améliorations apportées	11
7	Gestion de projet	12
8	Conclusion	12
9	Annexe	13
9.1	Autre fine-tuning de Whisper	13

1 Introduction

Dans le cadre de la formation de Master 2, parcours Intelligence Artificielle, module « Ingénierie R&D », nous avons travaillé sur un projet visant à améliorer la prononciation des modèles de synthèse vocale (TTS) à correspondance de flux conditionnel. Ce projet est réalisé en collaboration avec [Voxygen](#), une entreprise française spécialisée dans la synthèse vocale et les voix d’IA, basée en Bretagne. Il est également mené en partenariat avec le Laboratoire d’Informatique de l’Université du Mans ([LIUM](#)). Le projet s’est déroulé en parallèle des enseignements, selon des créneaux dédiés de deux semaines alternant avec les périodes de cours.

Réalisé en binôme, ce projet s’articule autour de deux phases principales. La première consiste à effectuer le fine-tuning d’un modèle de synthèse vocale, ainsi qu’à calculer des métriques afin d’évaluer ses performances. La seconde phase vise à concevoir un modèle d’apprentissage par renforcement dans le but d’améliorer la qualité de la voix synthétisée. Ces deux phases sont décrites en détail tout au long de ce rapport.

2 Présentation du projet

Ce projet s’inscrit dans le domaine du traitement du signal et de la synthèse vocale, et porte plus précisément sur les modèles de synthèse de la parole fondés sur le principe du *Conditional Flow Matching* (appariement de flux conditionnel), tels que le modèle ZipVoice [1]. L’objectif principal est d’analyser et d’améliorer la qualité de la parole synthétisée, en particulier du point de vue de la prononciation et de l’intelligibilité, à l’aide de métriques objectives telles que le *Character Error Rate* (CER) et le *Word Error Rate* (WER).

Une première partie du travail consiste à étudier l’impact du fine-tuning du modèle sur le corpus Blizzard2023 [2], afin d’évaluer les gains obtenus ainsi que les limites de cette adaptation en termes de spécialisation du modèle. Cette analyse sert de base à une seconde étape visant à explorer des méthodes d’optimisation plus avancées pour améliorer la qualité de génération vocale.

Dans un second temps, nous avons mis en place une approche basée sur l’apprentissage par renforcement. L’objectif de ce modèle est d’apprendre à sélectionner automatiquement, pour un texte cible donné, le prompt le plus pertinent (segment audio et transcription associés) afin de maximiser la qualité de la génération vocale. Cette qualité est évaluée à l’aide d’une métrique objective, le *Character Error Rate* (CER), que le modèle cherche à minimiser au cours de l’apprentissage.

3 Contexte et Problématique

Les modèles de génération d’audio présentent un biais important vers l’anglais lors de l’inférence. En effet, la majorité de ces modèles sont entraînés principalement sur des données anglophones, et la plupart des ressources disponibles sur Internet sont également en anglais. Par conséquent, lorsqu’on génère de l’audio en français, par exemple, le résultat est souvent influencé par un accent anglais, ce qui peut rendre certaines phrases difficiles à comprendre. Une solution consiste à affiner le modèle, déjà exposé à un petit pourcentage de données en français, afin de tirer parti de ses connaissances existantes et de l’adapter à la langue cible. Cette approche est particulièrement pertinente pour des cas d’utilisation spécifiques.

De plus, les modèles de génération d’audio fonctionnent généralement en *One-Shot* : en fournissant un court segment audio et un texte cible, le modèle reproduit le ton et la caractéristique vocale du segment pour générer un nouvel audio correspondant au texte cible. Il a été démontré [3, 4] que la qualité de l’audio généré dépend fortement du segment de référence (*prompt*) fourni. Il est donc crucial de déterminer à l’avance quel prompt utiliser pour un texte cible donné, ce

qui nécessite une approche non supervisée. C’est précisément ce que nous avons exploré dans ce projet.

4 Jeu de données et prétraitement

Nous avons travaillé sur le corpus *Blizzard2023* [5], un jeu de données audio français d’environ 53 heures, composé d’enregistrements provenant de deux locutrices distinctes : NEB et AD, avec respectivement environ 51 heures et 2 heures de parole.

Les données audio correspondent à des enregistrements de parole lue, produits par une seule locutrice à la fois, dans un style proche de la lecture de livres audio. Chaque enregistrement consiste en une lecture à voix haute, sans interaction ni échange dialogué.

Les données audio brutes sont constituées de 302 enregistrements de longue durée, tandis que les métadonnées décrivent des segments courts de 3 à 5 secondes. Un prétraitement a donc été réalisé afin de segmenter les fichiers audio longs en unités courtes alignées avec leurs métadonnées, permettant de constituer un jeu de données exploitable pour l’apprentissage. Le corpus résultant comprend 63 707 segments pour la locutrice NEB et 2 041 segments pour la locutrice AD.

Dans un second temps, les données ont été préparées pour la phase de génération. Le modèle ZipVoice [1] nécessite en entrée des fichiers `.tsv` respectant la structure suivante :

```
{wav_name}\t{prompt_transcription}\t{prompt_wav}\t{text}
```

Le modèle étant sensible aux caractères non conformes issus des conventions de transcription phonétique et typographique, un nettoyage itératif a été appliqué afin de supprimer ou normaliser les symboles incompatibles avec le format attendu (tels que {, }, ~, ^, -, §, etc.). Les segments provoquant des erreurs à la génération ou une dégradation notable de la qualité audio ont également été écartés. À l’issue de ce processus, le jeu de données utilisé pour la génération conserve un nombre de segments identique à celui du corpus de référence segmenté.

5 Modèles de Flow-matching – ZipVoice

La génération de voix par synthèse (TTS) a longtemps reposé sur des modèles basés sur la diffusion. L’idée de la diffusion est de transformer progressivement un bruit aléatoire en un signal audio réaliste en appliquant une série de petites étapes successives. Cependant, cette méthode présente un inconvénient majeur : le chemin entre le bruit initial et les données finales est souvent sinueux et complexe. Cela oblige le modèle à effectuer de nombreuses étapes de calcul lors de l’inférence, ce qui ralentit considérablement la génération de la voix.

5.1 Concept

Le *flow matching* est une méthode qui apprend à transformer progressivement un bruit aléatoire en données réalistes de manière continue et déterministe. Contrairement aux modèles de diffusion qui simulent un chemin aléatoire et complexe, le flow matching définit des trajectoires claires que chaque échantillon de bruit doit suivre pour devenir un échantillon de données. Ces trajectoires peuvent être choisies pour être géométriquement simples, presque droites (Figure??), ce qui facilite l’apprentissage et réduit le nombre d’étapes nécessaires pour générer un résultat.

L’idée principale est d’entraîner le modèle à prédire, pour chaque point de départ et à chaque instant, la direction et la vitesse à suivre pour transformer le bruit en données. Cette approche rend l’entraînement plus stable et l’inférence beaucoup plus rapide, car le modèle n’a pas besoin de simuler un processus aléatoire complexe. De plus, il est possible de guider les trajectoires selon des principes efficaces, comme le transport optimal, afin d’obtenir des chemins simples et directs entre le bruit et les données.

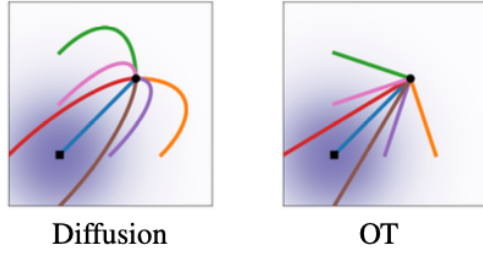


FIGURE 1 – Comparaison des trajectoires de génération. À gauche : la Diffusion suit des chemins courbes. À droite : le Flow Matching avec Transport Optimal (OT) [1]

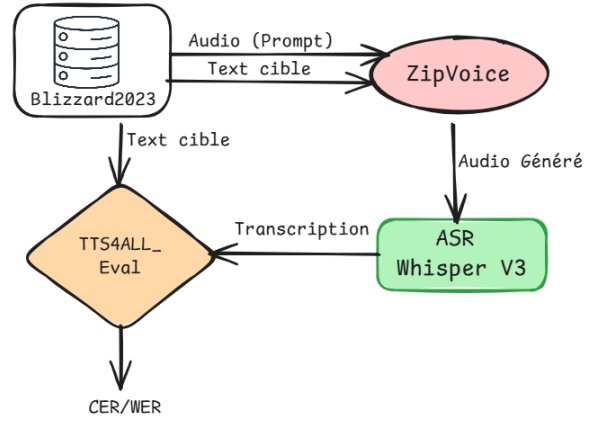


FIGURE 2 – Architecture du système d’évaluation.

ZipVoice [1] utilise cette approche pour la synthèse vocale ; la génération de voix dépend à la fois du texte cible à prononcer et d’un court exemple vocal de référence appelé *prompt*. Au lieu de produire l’audio pas à pas comme dans les modèles autoregressifs, il apprend un champ de vitesses conditionnel qui transforme des représentations simples, comme du bruit ou des latents, en spectrogrammes audio finaux en tenant compte du texte et du prompt.

Cela permet au modèle de reproduire la voix d’un locuteur à partir d’un seul exemple, de générer les spectrogrammes en parallèle pour accélérer l’inférence, et de réduire le nombre d’étapes avec une bonne qualité audio.

5.2 Évaluation des limites sur Blizzard2023

Pour évaluer la qualité de l’audio généré par ZipVoice sur le corpus Blizzard2023, nous avons réalisé des inférences directement sur ce corpus. Le modèle est pré-entraîné sur Emilia [6], un jeu de données d’environ 100 000 heures de parole, dont environ 95 % proviennent de l’anglais et du chinois. Comme indiqué précédemment, ZipVoice fonctionne en one-shot : pour chaque génération, le modèle reçoit un court extrait de référence (le *prompt*) ainsi que le texte cible. Dans nos expérimentations, le prompt et le texte cible sont identiques, ce qui simplifie l’évaluation mais ne reflète pas nécessairement les conditions d’utilisation dans la vie réelle.

5.2.1 Métriques

Il existe plusieurs catégories de métriques permettant d’évaluer la qualité audio, tels que la qualité perceptuelle, le réalisme, etc. Dans ce travail, l’analyse a porté principalement sur l’intelligibilité, c’est-à-dire la capacité à comprendre clairement les mots. Pour cela, nous avons utilisé deux métriques : le taux d’erreur sur les mots (*Word Error Rate*, WER) [7] et le taux d’erreur sur les caractères (*Character Error Rate*, CER) [7].

Ces métriques reposent sur la reconnaissance automatique de la parole (ASR). Dans ce cadre, nous avons utilisé le modèle *Whisper Large V3* [8]. L’ASR extrait la transcription des segments audio générés lors de l’inférence, puis, à l’aide d’un outil, nous calculons la différence entre cette transcription et le texte cible. Les métriques évaluent les erreurs de remplacement, de suppression et d’insertion, au niveau des caractères pour le CER et au niveau des mots pour le WER.

Techniquement, nous avons calculé les métriques à l’aide de `tts4all_eval` [9], un outil développé par des membres du LIUM pour l’évaluation de différentes catégories de métriques.

5.2.2 Résultats et analyse

Pour garantir la rigueur de l'évaluation, nous avons calculé les métriques selon un protocole en trois étapes :

1. **Mesure de la référence** : Estimation du biais intrinsèque de l'ASR Whisper sur le corpus original.
2. **Mesure de la baseline** : Évaluation du modèle ZipVoice pré-entraîné (Zero-Shot).
3. **Mesure du modèle affiné** : Quantification des gains apportés par nos différentes stratégies de fine-tuning.

Afin d'optimiser les ressources de calcul, l'évaluation a été réalisée sur un sous-ensemble de test stratifié représentant 10 % du corpus (le calcul sur l'intégralité du volume excédant 72 heures de traitement).

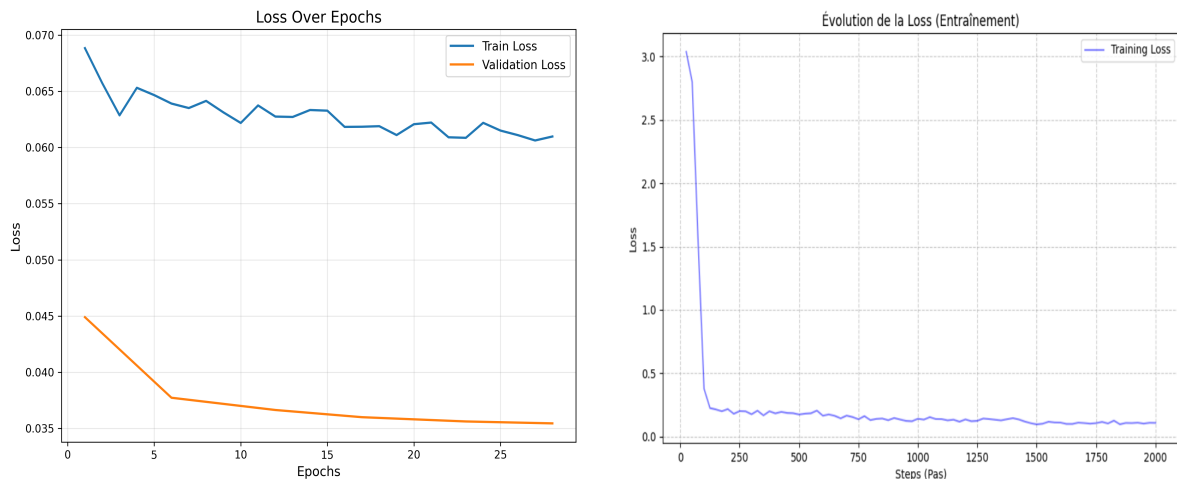


FIGURE 3 – Dynamique d'apprentissage. À gauche : convergence stable de la perte pour ZipVoice (MSE). À droite : perte d'entraînement de Whisper.

Le Tableau 1 synthétise les résultats. La colonne de droite (*Whisper V3 Large Fine-tuned LoRA*) constitue notre métrique de référence fiable.

5.3 Analyse du Fine-tuning de ZipVoice

Le processus de fine-tuning de ZipVoice (122 millions de paramètres) a fait l'objet de deux itérations distinctes afin d'identifier la stratégie de convergence optimale.

5.3.1 Échec de l'Approche Généraliste (V1)

La première tentative (voir ligne *ZipVoice FT V1* du tableau) consistait à entraîner le modèle sur le mélange des deux locuteurs (AD et NEB) pendant 50 000 itérations. Cette stratégie a conduit à une dégradation majeure des performances ($WER > 60\%$). Trois facteurs expliquent cet échec :

- **Incohérence timbrale** : La tentative de modéliser simultanément une voix masculine et féminine a empêché la convergence vers une cible acoustique stable.
- **Sur-apprentissage du bruit** : Bien que la fonction de perte (MSE) semblait faible (Figure 3, gauche), le modèle a mémorisé le bruit de fond des enregistrements au détriment de la prononciation.
- **Oubli Catastrophique** : La durée excessive de l'entraînement a effacé les connaissances phonétiques généralistes acquises lors du pré-entraînement.

TABLE 1 – Comparaison des métriques CER / WER sur le sous-ensemble de test. On observe l’échec de l’approche mixte (V1) et le succès de la spécialisation (V2).

Configuration	Mesure : Whisper Base		Mesure : Whisper FT LoRA	
	CER	WER	CER	WER
Référence (Ground Truth)				
AD	0.0798	0.2272	0.0072	0.0257
NEB	0.1219	0.3103	0.0077	0.0265
ZipVoice (Pré-entraîné)				
AD	0.2802	0.6426	0.1484	0.3194
NEB	0.4914	0.8827	0.1159	0.2668
ZipVoice FT V1 (Mixte - 50k it)				
AD	0.3382	0.7635	0.2928	0.6529
NEB	0.3413	0.7567	0.2795	0.6390
ZipVoice FT V2 (NEB - 10k it)				
NEB	0.0804	0.1923	0.0690	0.1784

5.3.2 Succès de la Spécialisation (V2)

La seconde approche (V2) a corrigé ces biais par une spécialisation stricte sur la locutrice cible (NEB) et l’application d’un arrêt précoce (*Early Stopping*) à 10 000 itérations. Comme le montre le Tableau 1, cette stratégie divise le taux d’erreur par trois (WER de 17,84 %). Le modèle conserve la structure grammaticale du pré-entraînement tout en adoptant fidèlement le timbre de la locutrice cible.

5.3.3 Dynamique de la Perte

L’évolution de la fonction de perte oscille faiblement entre 0,058 et 0,063. Cette stabilité apparente s’explique par la compression des caractéristiques spectrogrammes (Log-Mel) et la proximité initiale du modèle pré-entraîné avec l’optimum. Nous notons une perte de validation systématiquement inférieure à la perte d’entraînement. Ce phénomène est dû au mécanisme de *Conditional Drop* de ZipVoice : durant l’entraînement, une partie du conditionnement textuel est masquée pour robustifier le modèle, une contrainte levée lors de la validation, simplifiant mécaniquement la tâche de prédiction.

5.4 Validation de la Métrique : Fine-tuning de Whisper avec LoRA

Pour obtenir les mesures précises affichées dans la colonne de droite du Tableau 1, nous avons dû adapter notre outil de mesure. Le modèle Whisper de base présentait en effet un biais important sur ce corpus (WER > 20 %).

Nous avons appliqué une méthode d’adaptation de bas rang (**LoRA**) ciblant les projections d’attention (**q_proj**, **v_proj**) avec un rang $r = 32$ et un alpha $\alpha = 64$. Cette configuration permet de spécialiser le modèle sur la prosodie littéraire du corpus Blizzard sans altérer ses capacités de reconnaissance générales. Le pré-traitement textuel a été strictement aligné sur la norme Unicode NFC pour garantir la cohérence entre la fonction de perte et le calcul du WER. Cette adaptation a permis de réduire l’erreur de référence à moins de 3 %, validant la fiabilité des mesures présentées.

5.5 Limitations

Le calcul du CER et du WER est très long : plusieurs jours seraient nécessaires pour l'évaluer sur l'ensemble du corpus. Nous avons donc utilisé un sous-ensemble de 5 % pour obtenir une estimation fiable tout en limitant le coût computationnel, avec plusieurs itérations réalisées pour consolider les résultats.

6 Optimisation de la sélection de prompt par apprentissage par renforcement

Comme mentionné précédemment, ZipVoice est un modèle *one-shot*. Il prend en entrée un texte cible à générer ainsi qu'un court extrait audio de référence (le *prompt*), accompagné de sa transcription, à partir duquel la voix du locuteur est clonée.

Des travaux [3, 4] ont montré que la qualité du segment audio généré dépend fortement du choix du segment de référence. Ainsi, afin de produire une synthèse vocale de la meilleure qualité possible, il est essentiel de sélectionner de manière appropriée le prompt pour un texte cible donné, d'autant plus que la transcription du prompt varie en fonction du texte cible.

De notre côté, nous avons également observé cet effet expérimentalement. Nous avons sélectionné un texte cible aléatoire du corpus et évalué la génération à l'aide de dix prompts différents, dont l'un correspondait exactement au texte cible. Les résultats montrent que lorsque le prompt et le texte cible sont identiques, le CER est nul ($\text{CER} = 0.00$), tandis que l'utilisation d'un autre prompt peut entraîner une dégradation significative des performances, avec un CER pouvant atteindre 0.48.

6.1 Motivation

Pour tenter de résoudre ce problème, c'est-à-dire identifier le meilleur prompt de transcription et le segment WAV correspondant pour un texte cible donné, nous ne disposons pas de données annotées. Une approche possible consiste à utiliser l'apprentissage par renforcement [10], car ce type de méthode permet de traiter ce problème en l'absence d'annotations directes.

L'approche d'apprentissage par renforcement que nous avons mise en place est le *contextual bandit* [10], dans lequel le modèle, appelé *agent*, évolue dans un environnement et doit choisir une action en fonction d'une récompense qu'il reçoit. La récompense est déterminée par une métrique (expliquée ci-après). Si celle-ci est élevée, l'agent apprend à reproduire ce comportement ; si la récompense est faible, le modèle apprend peu. Il convient de noter que l'agent commence par une initialisation aléatoire. Au fil du temps, il identifie progressivement les motifs (*patterns*) qui permettent de sélectionner le meilleur prompt pour un texte cible.

6.2 Prétraitement

Nous avons créé une base de données vectorielle pour tous les segments du corpus Blizzard2023 [2] de la locutrice NEB en utilisant SONAR [11], qui prend le segment audio avec sa transcription et produit en sortie un vecteur de 1024 dimensions. Ensuite, nous avons appliqué une Analyse en Composantes Principales (PCA) pour compresser ces vecteurs à 100 dimensions, pour des raisons de simplicité et d'efficacité computationnelle.

6.3 Architecture du pipeline

Pour rendre le processus clair et compréhensible, nous avons divisé le pipeline en étapes illustrées comme suit :

1. **Encoder le texte cible** : en utilisant SONAR pour obtenir un vecteur d'embedding de taille 1024 dimensions. Cette étape permet d'extraire des caractéristiques sémantiques et linguistiques qui seront interprétables par le modèle dans les étapes suivantes.
2. **Exécuter le modèle agent** : le modèle consiste en un réseau de neurones profond à deux têtes avec des connexions résiduelles qui prend les 1024 dimensions de l'Étape 1 et prédit :
 - **Tête d'action (policy)** : avec un vecteur de taille 256 dimensions en sortie représentant l'embedding du prompt à sélectionner
 - **Tête de valeur** : un scalaire estimant la récompense attendue pour guider l'apprentissage.

L'architecture mise en place est la suivante :

- **Projection d'entrée** : $1024 \rightarrow 512$, avec LayerNorm, GELU comme fonction d'activation, et Dropout ($p = 0.15$) pour stabiliser le flux de gradients, notamment lors des récompenses rares.
- **Blocs résiduels** : 4 blocs résiduels. Chaque bloc implémente :

$$\text{ResidualBlock}(x) = x + \text{MLP}(\text{LayerNorm}(x))$$

- où le MLP utilise un facteur d'expansion de 4 : $512 \rightarrow 2048 \rightarrow 512$ avec activation GELU. Cette architecture permet un meilleur flux de gradient et une capacité représentationnelle accrue.
- **Tête d'action** : $512 \rightarrow 512 \rightarrow 256$, avec normalisation L2 pour aligner le vecteur produit (embedding du prompt à sélectionner) sur les embeddings déjà sauvegardés. La normalisation L2 contraint ce vecteur à une normalisation L2 afin de faciliter le calcul de la similarité cosinus.
 - **Tête de valeur améliorée** : architecture séparée pour prévenir l'interférence avec la politique : $512 \rightarrow 256 \rightarrow 64 \rightarrow 1$ avec activation Sigmoid pour contraindre la sortie à $[0, 1]$ correspondant à la plage de récompenses. Une projection intermédiaire supplémentaire améliore la capacité d'estimation.

Par ailleurs, afin d'optimiser les capacités d'apprentissage du modèle et de garantir la robustesse de l'architecture, nous avons implémenté les stratégies d'exploration et de régularisation suivantes :

- **Exploration ϵ -greedy** : C'est une technique pour que le modèle ne fasse pas toujours la même décision. Dans la plupart des cas d'entraînement, le modèle choisit la meilleure action qu'il connaît à partir de ses prédictions passées. Mais, avec une petite probabilité ϵ , il choisit une action au hasard (exploration) afin de prendre de nouvelles décisions et d'apprendre plus. La valeur de ϵ (0.5) démarre à une valeur élevée au début afin que le modèle explore plus au départ, puis elle diminue exponentiellement quand le modèle gagne en confiance.
- **Normalisation adaptative des récompenses** : Afin de stabiliser l'apprentissage, nous avons adopté une stratégie de normalisation basée sur la moyenne des récompenses. Cette approche calcule l'avantage comme la différence entre la récompense obtenue et la moyenne des récompenses observées dans le replay buffer (voir après), ce qui va centrer les signaux d'apprentissage autour de zéro. En complément, une normalisation par l'écart-type des récompenses est appliquée lorsque plusieurs échantillons sont disponibles, assurant une stabilité accrue et une convergence plus fiable de l'algorithme d'apprentissage par renforcement.
- **Pénalité de diversité avec décroissance** : cette pénalité sert à forcer la diversité en décourageant le modèle de réutiliser systématiquement les mêmes prompts, empêchant ainsi le système de s'enfermer dans une solution unique et répétitive (phénomène de mode collapse).

3. **Mesurer la similarité** : calculer la similarité entre la prédiction (vecteur de 256 dimensions normalisé L2) et l'ensemble des vecteurs de la base de données vectorielle FAISS, afin de sélectionner le prompt le plus proche selon la similarité cosinus, puis de retourner son audio et sa transcription associés. La base vectorielle contient 63,700 prompts, réduits de 1024 à 256 dimensions par ACP pour faciliter l'apprentissage.
4. **Générer l'audio avec ZipVoice** : qui prend la transcription et le segment audio du prompt sélectionné à l'étape précédente ainsi que le texte cible de l'étape 1 et génère un nouveau segment audio par clonage de voix en *one-shot*.
5. **Mesurer le Character Error Rate (CER)** : en utilisant Whisper Large v3 [8], un modèle de reconnaissance automatique de la parole (*Automatic Speech Recognition*, ASR). L'idée est d'obtenir la transcription du segment de référence provenant du corpus et du segment qui vient d'être généré par ZipVoice en utilisant la prédiction comme prompt. Le CER mesure la différence entre ces deux transcriptions caractère par caractère selon la distance d'édition de Levenshtein, avec une erreur plus basse indiquant une meilleure qualité de synthèse vocale.
6. **Calculer la récompense pondérée** : pour guider l'agent vers de meilleurs choix. Si la récompense est élevée, l'agent va apprendre beaucoup d'informations de cet exemple et les poids du modèle vont se mettre à jour fortement dans la direction de cette action optimale. Si la récompense est faible, le modèle va à peine apprendre de cet exemple et son état va rester approximativement le même. nous avons défini la récompense par la formule :

$$R = \max(0, 1 - \text{CER})$$

Cette formulation normalise la récompense sur l'intervalle $[0, 1]$. Ainsi, une transcription parfaite ($\text{CER} = 0$) génère une récompense maximale de 1.0, tandis qu'un taux d'erreur supérieur ou égal à 100% entraîne une récompense nulle, ce qui va pénaliser les générations de faible qualité.

7. **Mise à jour du modèle avec Policy Gradient et apprentissage contrastif** : dans cette étape finale, le modèle effectue une mise à jour des gradients. nous avons deux types de gradients. le gradient REINFORCE ajuste la politique directement en fonction de la récompense faisant converger le modèle vers les zones à fort rendement, alors que le gradient contrastif agit comme un régularisateur structurel. Il force l'espace des embeddings prédit en rapprochant la prédiction de l'action optimale ainsi qu'éloigner des autres actions concurrentes du batch, pour accélérer la convergence.

L'objectif global est défini par la minimisation de la fonction de perte composite suivante :

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{PG}}}_{\text{Politique}} + \underbrace{0.3 \cdot \mathcal{L}_{\text{InfoNCE}}}_{\text{Contraste}} + \underbrace{\mathcal{L}_{\text{value}}}_{\text{Valeur}} \underbrace{-\alpha \cdot H(\text{pred})}_{\text{Entropie}} + \underbrace{\beta \cdot \text{Pen}_{\text{div}}}_{\text{Diversité}} \quad (1)$$

Cette équation orchestre les dynamiques suivantes :

- **Guidage par l'avantage ($\mathcal{L}_{\text{PG}}$)** : Utilise l'avantage normalisé ($A = R - R_{\text{baseline}}$) pour renforcer les actions performantes par rapport à une moyenne mobile historique.
- **Structuration contrastive ($\mathcal{L}_{\text{InfoNCE}}$)** : Rapproche la prédiction des exemples positifs du batch tout en éloignant des exemples négatifs pour accélérer la convergence.
- **Régularisation (α, β)** : Combine l'entropie et une pénalité de diversité pour prévenir l'effondrement du mode (*mode collapse*) et garantir l'exploration.

Stratégie d'entraînement : L'apprentissage s'appuie sur un *Replay Buffer* couplé à un mécanisme d'échantillonnage « conscient de la diversité » (*diversity-aware sampling*). Cette approche sélectionne activement des expériences variées pour briser les corrélations temporelles et prévenir l'oubli catastrophique.

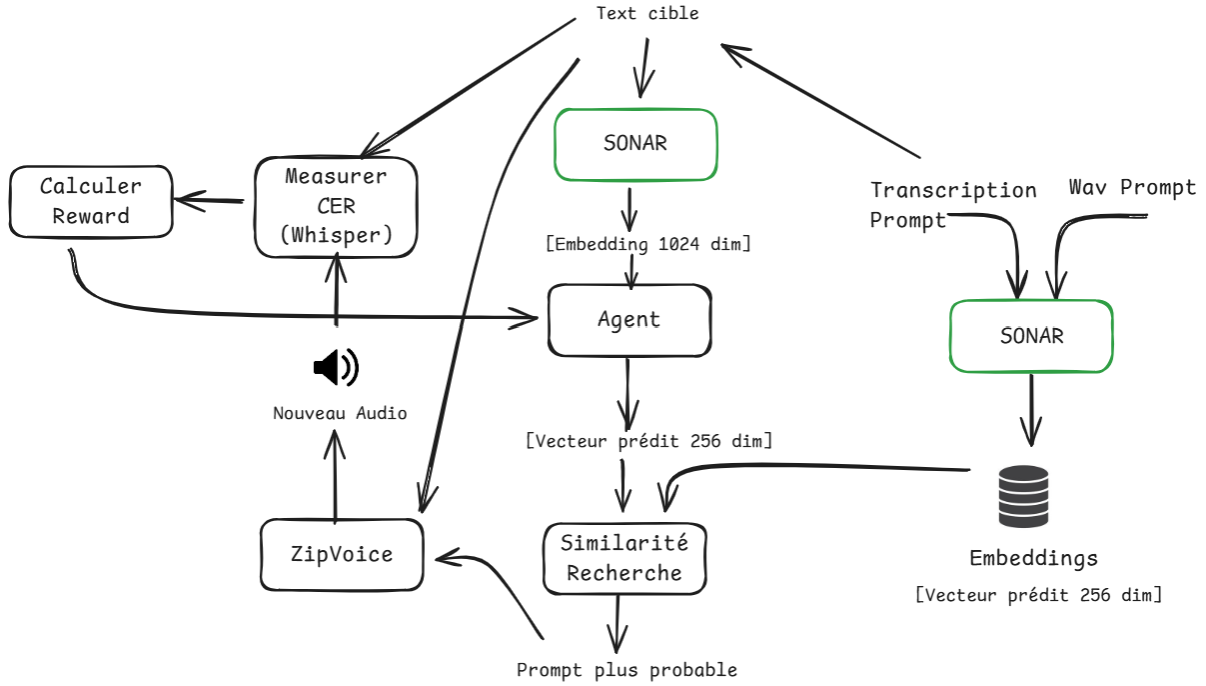


FIGURE 4 – Architecture du système de sélection optimale de prompts par apprentissage par renforcement pour le modèle TTS ZipVoice

6.4 Résultats et analyse

Afin de reprendre les cette partie claire et lisible nous illustrerons les résultats obtenus :

- * La courbe de loss (voir figure 6) diminue au fil du temps (axe horizontal : nombre de phrases traitées), avec de petites oscillations attendues lors d'un apprentissage progressif à partir d'exemples variés et d'essais exploratoires. La valeur de loss stagne après 500 phrases, indiquant que l'apprentissage atteint un palier. Les valeurs de loss sont négatives, ce qui est normal car la composante entropie dans l'équation (1) domine les autres termes.
- * Le modèle présente un biais significatif en faveur des prompts courts, les prompts de sept mots ou moins étant sélectionnés 3,57 fois plus fréquemment que celles de douze mots ou plus. Cette préférence se traduit également dans les performances : les prompts courts obtiennent un CER moyen de 0,2906 ($n = 357$), tandis que les prompts longues présentent un CER moyen de 0,4019 ($n = 100$).
- * Les phrases extrêmement courtes (texte cible de 1 à 3 mots) présentent des erreurs élevées, avec un CER moyen souvent supérieur à 0,7. Plus précisément, pour les phrases très courtes (1 à 3 mots), le CER moyen est de 0,4428, tandis que pour les phrases longues (11 mots ou plus), le CER moyen est de 0,2693.
- * Vu que le modèle présente une préférence pour les phrases courtes, nous avons également mené une autre expérience (sur 194 phrases, l'expérience s'est arrêtée en raison d'une erreur) dans laquelle nous avons éliminé les prompts de moins de 6 mots. Cette approche montre une légère amélioration globale des performances par rapport au pipeline original, avec un CER moyen de 0,3599 contre 0,3653 (soit une baisse de 1,5%). voir 7
- * Une autre expérience a été menée (sur 500 phrases) consistant à limiter le nombre de prompts disponibles, passant d'environ 63 000 à 5 000. L'hypothèse était que l'espace complexe de 63 000 prompts pouvait dégrader les résultats en rendant la sélection plus difficile. Cependant, les résultats montrent une dégradation : le CER moyen est passé de

0,3168 à 0,3656, ce qui confirme que la réduction de l'index réduit la performance globale du modèle. voir 7

6.5 Limitations

La pipeline proposée est très chronophage. En effet, le traitement d'une seule phrase implique successivement la génération d'un embedding sémantique par SONAR, la synthèse d'un segment audio par ZipVoice, puis sa transcription automatique par Whisper. Cette chaîne de traitements rend chaque itération particulièrement lente. En conséquence, l'entraînement du modèle est long : le traitement d'environ 600 phrases a nécessité près de 48 heures.

Par ailleurs, pour obtenir une vision claire de la convergence du modèle et de sa stabilité à long terme, il serait nécessaire de l'entraîner sur plusieurs dizaines de milliers de phrases. Une telle expérimentation dépasse toutefois le cadre et les contraintes temporelles de ce projet.

6.6 Écueils rencontrés et améliorations apportées

Au cours du développement, plusieurs problèmes méthodologiques ont été identifiés lors des premières expérimentations sur 1000 phrases :

- **Architecture trop simple** : La première version se limitait à deux couches linéaires ($1024 \rightarrow 512 \rightarrow 100$) sans connexions résiduelles ni normalisation. Elle présentait des signes clairs de sur-apprentissage dès les premières itérations. L'introduction de 4 blocs résiduels avec LayerNorm a résolu ce problème.
- **Embeddings sous-dimensionnés** : Les embeddings de 100 dimensions causaient une perte d'information sémantique, limitant la discrimination entre prompts. L'augmentation à 256 dimensions a amélioré significativement la qualité des prédictions.
- **Exploration aléatoire non structurée** : L'échantillonnage uniforme dans R^{256} générait des actions aberrantes hors de la distribution réelle. L'exploration guidée par centroïdes K-means avec perturbation gaussienne ($\sigma = 0.1$) maintient désormais l'exploration dans des régions sémantiquement plausibles.
- **Mode collapse** : Le modèle convergeait vers moins de 10 prompts distincts sur 1000 disponibles. Une pénalité de diversité avec décroissance exponentielle ($\alpha = 0.99$) force le maintien d'une distribution d'actions équilibrée.
- **Oubli catastrophique** : L'entraînement en ligne sans mémoire écrasait les connaissances acquises, causant oscillations et stagnation. Un replay buffer de capacité 5000 avec échantillonnage diversifié stabilise l'apprentissage et brise les corrélations temporelles.

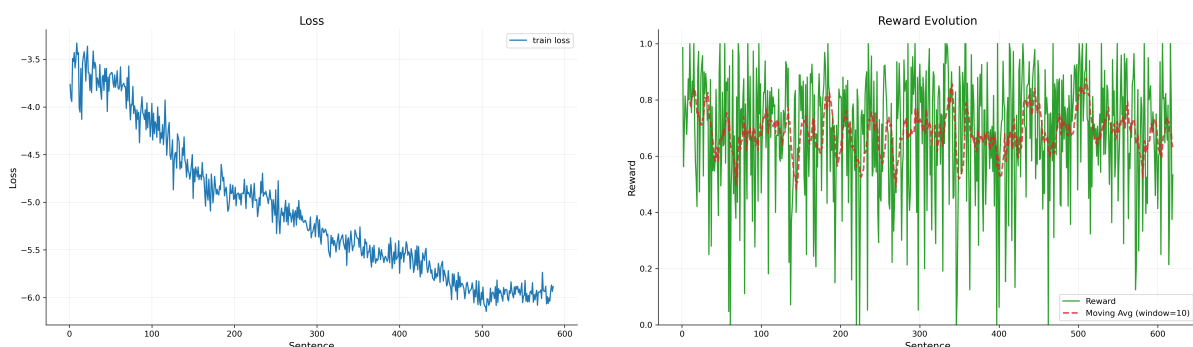


FIGURE 5 – Perte d'entraînement à gauche, et évolution de la récompense au cours du temps à droite.

7 Gestion de projet

Le cadre de ce projet se divise en deux phases principales, comme mentionné précédemment. Durant les deux premières périodes et demie du projet (du 23 septembre au 3 octobre, du 20 octobre au 31 octobre et du 24 novembre au 28 novembre), nous nous sommes concentrés sur la première partie : la prise en main de ZipVoice et de Whisper, ainsi que les tests d'inférence et de fine-tuning.

Ensuite, pendant une période et demie (du 2 décembre au 5 décembre et du 19 janvier au 30 janvier), nous avons travaillé sur la partie apprentissage par renforcement.

Par ailleurs, toutes les expérimentations ont été réalisées sur le cluster étudiant de la faculté afin de bénéficier de ressources GPU et de pouvoir lancer des scripts sur de longues durées. Nous avons utilisé VS Code comme éditeur de code principal.

La communication avec les encadrants s'est faite via Mattermost, tandis que Discord a été utilisé pour discuter de l'avancement du projet et partager rapidement des extraits de code au sein de l'équipe.

8 Conclusion

Ce projet nous a permis d'explorer différentes méthodes pour améliorer la prononciation d'un modèle de synthèse vocale en français. Les expériences montrent que le fine-tuning léger avec LoRA est plus efficace que l'ajustement complet du modèle, tout en évitant une dégradation des connaissances déjà acquises. L'utilisation de l'apprentissage par renforcement pour sélectionner automatiquement les prompts s'est également révélée utile, en soulignant l'importance du choix du prompt sur la qualité de la synthèse. Malgré des ressources de calcul limitées, ce travail met en évidence plusieurs pistes prometteuses pour améliorer conjointement les modèles TTS et les stratégies de sélection de prompts.

9 Annexe

- Vous pouvez consulter le code du pipeline d'apprentissage par renforcement à l'adresse suivante :
<https://github.com/NASSWIEL/ZipBandit>
- Le code de fine-tuning de ZipVoice et de Whisper est disponible sur le cluster de la faculté, dans le répertoire :
`/info/raid-etu/m2/s2405959/V02/`
- Pour écouter des exemples générés par ZipVoice dans différents états, consultez :
https://github.com/NASSWIEL/CFT-TTS_overview
- Pour consulter le corpus segmenté ainsi que les différents niveaux de génération de ZipVoice pour les deux voix, veuillez consulter le répertoire :
`/info/corpus/Blizzard2023segmented/`

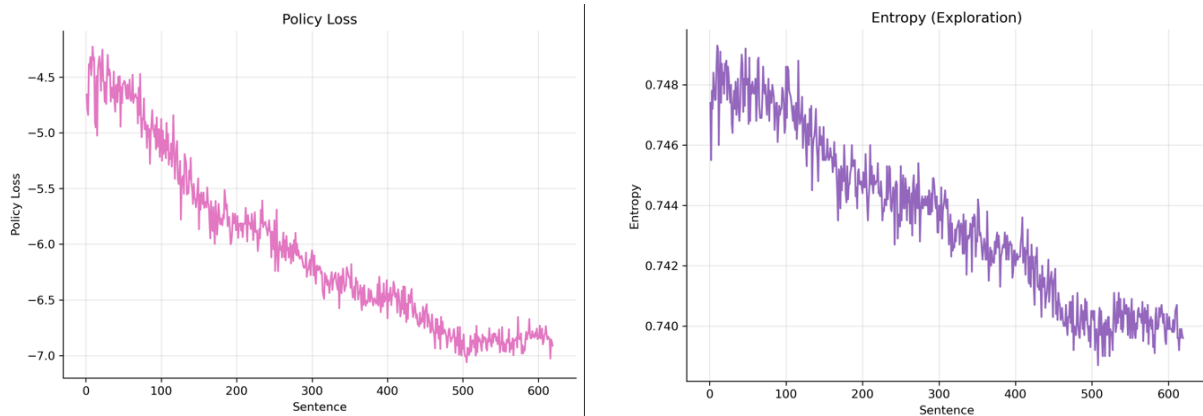


FIGURE 6 – Métriques d'apprentissage par renforcement : évolution de la perte de la politique à gauche, et l'évolution de l'entropie (taux d'exploration) à droite, en fonction du nombre de phrases traitées.

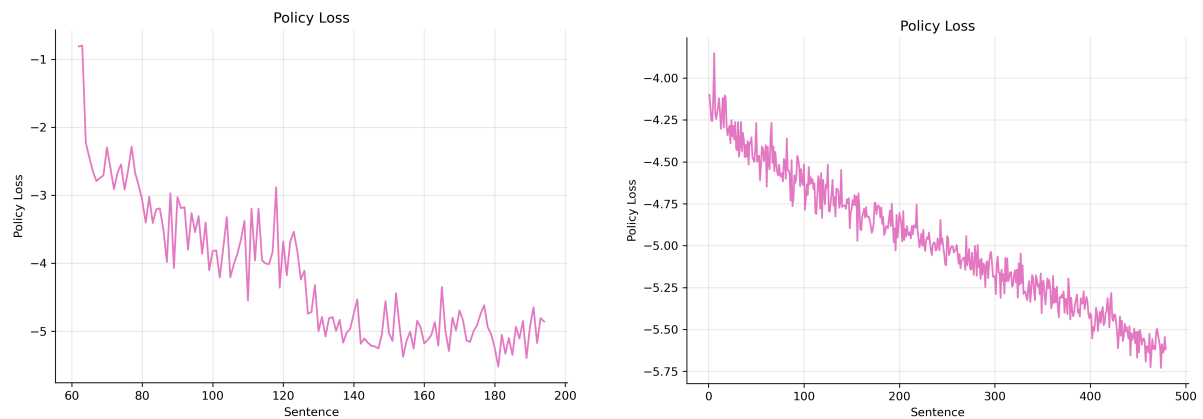


FIGURE 7 – Évolution de la perte de la politique (*policy loss*). À gauche : expérience avec filtrage des prompts (longueur minimale de 6 mots). À droite : expérience avec un espace de recherche restreint à 5 000 prompts.k

9.1 Autre fine-tuning de Whisper

Nous avons également testé un fine-tuning complet de Whisper (mise à jour de l'ensemble des poids), mais cette approche ne s'est pas révélée optimale. Le fine-tuning a été effectué sur

50 000 itérations, cependant les résultats obtenus sont inférieurs à ceux du fine-tuning avec LoRA, comme indiqué dans la Table ??.

Pour la locutrice NEB, le fine-tuning complet atteint 0,69 / 0,60 (WER / CER) sur la référence, contre 0,31 / 0,08 (WER / CER) avec LoRA, soit une réduction d'environ moitié des taux d'erreur. La même tendance est observée sur l'autre sous-ensemble du corpus. En conséquence, nous avons conservé uniquement les résultats issus du fine-tuning avec LoRA.

Références

- [1] k2-fsa, “Zipvoice : Fast and high-quality zero-shot text-to-speech with flow matching,” <https://github.com/k2-fsa/ZipVoice>, 2025, accessed : 2026-01-09.
- [2] O. Perrotin, B. Stephenson, S. Gerber, and G. Bailly, “The blizzard challenge 2023,” in *18th Blizzard Challenge Workshop*, 2023, available from the ISCA Archive, describing the released French TTS corpus for Blizzard Challenge 2023. [Online]. Available : https://www.isca-archive.org/blizzard_2023/perrotin23_blizzard.html
- [3] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, L. He, S. Zhao, and F. Wei, “Neural codec language models are zero-shot text to speech synthesizers,” *arXiv preprint arXiv :2301.02111*, 2023, <https://arxiv.org/abs/2301.02111>.
- [4] Y. Jia, Y. Zhang, R. J. Weiss, Q. Wang, J. Shen, F. Ren, Z. Chen, P. Nguyen, R. Pang, I. Lopez Moreno, and Y. Wu, “Transfer learning from speaker verification to multispeaker text-to-speech synthesis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, <https://papers.nips.cc/paper/7700-transfer-learning-from-speaker-verification-to-multispeaker-text-to-speech-synthesis>.
- [5] O. Perrotin, B. Stephenson, S. Gerber, G. Bailly, and S. King, “Refining the evaluation of speech synthesis : A summary of the blizzard challenge 2023,” *Computer Speech and Language*, vol. 90, p. 101747, 2025, early online 8 Nov 2024. [Online]. Available : <https://www.research.ed.ac.uk/en/publications/refining-the-evaluation-of-speech-synthesis-a-summary-of-the-bliz>
- [6] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi, Y. Wang, K. Chen, P. Zhang, and Z. Wu, “Emilia : An extensive, multilingual, and diverse speech dataset for large-scale speech generation,” in *Proc. of IEEE Spoken Language Technology Workshop (SLT)*, 2024.
- [7] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech : An asr corpus based on public domain audio books,” *Proc. ICASSP*, 2015.
- [8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” *arXiv preprint*, 2022, arXiv :2212.04356. [Online]. Available : <https://arxiv.org/abs/2212.04356>
- [9] LIUM – Université du Mans, “Tts4all evaluation repository,” https://git-lium.univ-lemans.fr/jsalt2025/wp1/tts4all_eval/, 2025, consulté le 23 janvier 2026.
- [10] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 661–670, <https://doi.org/10.1145/1772690.1772758>.
- [11] P. Duquenne, H. Schwenk, and B. Sagot, “SONAR : sentence-level multimodal and language-agnostic representations,” *arXiv preprint*, 2023, <https://arxiv.org/abs/2308.11466>.